CoopInfer: Host–Device Cooperative Inference Scheduling

A strategy-driven prototype for DAG partitioning and schedule visualization

CoopInfer Project

June 24, 2026

Motivation

- Embodied AI pipelines often run as DAGs of perception, fusion, and control operators.
- Some operators must stay on the robot-side device, while others may run on an edge host.
- The scheduling problem is not only graph partitioning:
  - cross-device edges incur network transfer cost;
  - each side has a finite execution queue;
  - end-to-end latency constraints can make otherwise low-loss solutions infeasible.
- CoopInfer is a desktop prototype for editing, solving, and visualizing these schedules.

System Overview

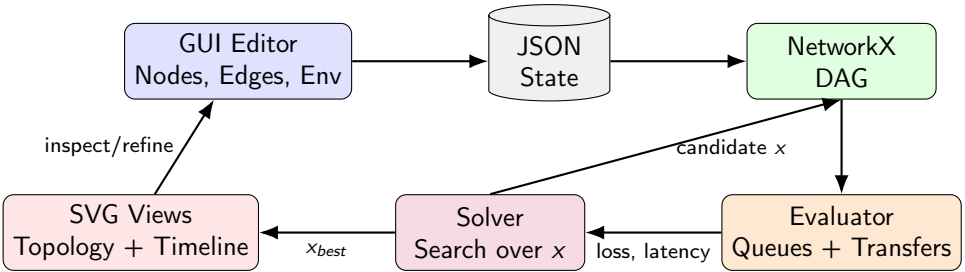
Inputs

- DAG nodes with device/host compute costs.
- Edges with transfer sizes.
- Bandwidth, latency, objective weight.
- Serialized network model with optional transfer batching.
- Optional E2E latency limit.

Outputs

- Node placement: $x_i = 0$ for DEVICE, $x_i = 1$ for HOST.
- End-to-end latency and device utilization.
- SVG topology and profiler-style schedule timeline.

Prototype Loop

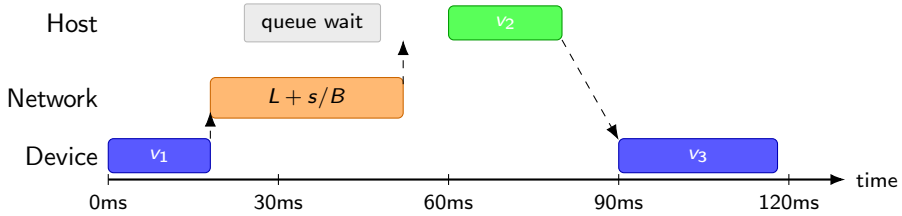


The prototype keeps editing, solving, physical evaluation, and visualization in one reproducible workflow.

Data Model

- In memory, topology is stored in a `networkx.DiGraph`.
- Node attributes:
  - `id`, `name`, `c_dev`, `c_host`, `fixed_dev`, `x`.
  - Optional input cadence fields: `source_period_ms`, `source_phase_ms`.
- Edge attributes:
  - `source`, `target`, `size` in MB.
- Environment fields:
  - `bandwidth`, `latency`, `weight_latency`, `latency_limit`, `batch_transfers`.
- JSON load/save keeps experiments reproducible and shareable.

Queue-Aware Execution



A node starts only after all data is ready and its assigned worker queue is available.

Evaluator: Data Dependency + Resource Queues

For each node i in topological order:

$$DataReady_i = \max\left(release_i(f), \max_{j \in pred(i)} \left[F_j + \mathbb{I}(x_j \neq x_i)\left(L + \frac{s_{ji}}{B}\right)\right]\right)$$

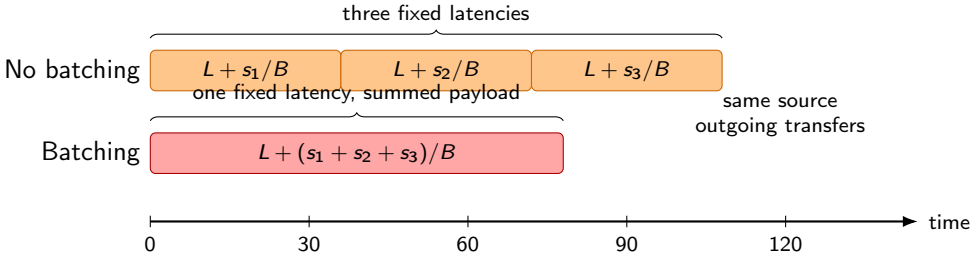
where input/source nodes may use $release_i(f) = phase_i + f \cdot period_i$ for frame f. Source nodes are zero-duration external events: they do not occupy the device or host worker queues, and later releases may occur while older frames are still being processed downstream. The evaluator then waits for the selected worker queue:

$$S_i = \begin{cases} \max(DataReady_i, time\_dev\_ready), & x_i = 0 \\ \max(DataReady_i, time\_host\_ready), & x_i = 1 \end{cases}$$

$$F_i = S_i + (1 - x_i)c_i^{dev} + x_i c_i^{host}$$

The E2E latency is $\tau = \max_i F_i$.

Network Batching Illustration



Batching reduces repeated fixed latency, but it does not allow network transfers to overlap.

Network Model: Serialized Transfers and Batching

- Cross-device transfers share one network worker timeline:

$$time\_network\_ready$$

- A transfer can start only after its source node finishes and the network worker is idle.
- Without batching, each cross-device edge is scheduled independently:

$$T_{edge} = L + \frac{s}{B}$$

- With `batch_transfers=true`, multiple outgoing cross-device edges from the same source node are sent as one transaction:

$$T_{batch} = L + \frac{\sum_k s_k}{B}$$

- This preserves non-overlap while avoiding repeated fixed latency for successive outgoing transfers.

Objective and Feasibility

The solver minimizes a weighted objective:

$$J = w \cdot \frac{\tau - \tau_{min}}{\tau_{max} - \tau_{min}} + (1 - w)\left(1 - \frac{\sum_i (1 - x_i) c_i^{dev}}{\sum_i c_i^{dev}}\right)$$

Latency limit

A positive `latency_limit` acts as a hard feasibility constraint:

$$\tau \leq latency\_limit$$

Candidates above the limit are rejected before objective comparison.

Solver Modes

Mode	Use case	Behavior
Auto	Default	Enumerates if ≤ 12 free nodes; otherwise random search
Enumerate	Small DAGs	Tests every assignment and returns the feasible global best
Random Search	Larger DAGs	Perturbs placements and keeps the best feasible candidate
Simulated Annealing	Escaping local minima	Accepts some worse moves according to temperature

All modes enforce fixed-device nodes and the E2E latency cap.

GUI Workflow

1. Edit nodes, names, compute costs, fixed-device flags.
2. Edit edges and transfer sizes in MB.
3. Configure bandwidth, latency, network batching, objective weight, solver mode, iterations, and E2E cap.
4. Click **Solve**.
5. Inspect metrics, topology placement, and profiler-style timeline.

Validation catches duplicate node IDs, unknown edge endpoints, and non-DAG topologies.

Visualization Design

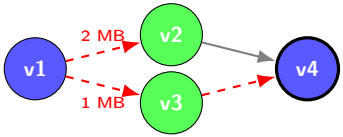
Topology SVG

- Blue: DEVICE.
- Green: HOST.
- Bold node border: fixed on DEVICE.
- Red dashed edge: cross-device transfer.
- Gray solid edge: local dependency.

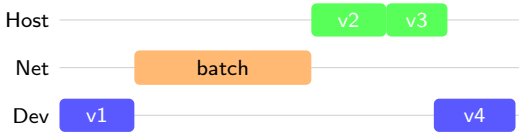
Timeline SVG

- Host, Transfer, Device lanes.
- Operator bars use computed start/finish times.
- Transfer bars use actual evaluator records, including serialized and batched transfers.
- Browser-native scroll and scale controls.

Topology and Timeline Reading Guide

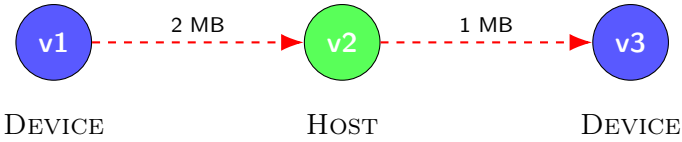


Topology: colors show placement; edge style shows communication



Timeline: bars show actual evaluator start/finish times

Example DAG



The evaluator produces a queue-aware schedule rather than an ideal infinite-parallel critical path.

Current Implementation Stack

- Python 3.9+.
- PyQt6 for desktop controls and layout.
- PyQt6-WebEngine for SVG topology and timeline rendering.
- NetworkX for DAG storage, topological sort, and validation.
- Pytest coverage for evaluator, JSON IO, solver modes, and latency-limit feasibility.
- Launch commands: `python -m coopinfer` or `python -m coopinfer.app`.

Next Steps

- Add richer solver diagnostics: rejected candidates, constraint slack, convergence history.
- Add exportable SVG/PNG reports for topology and timeline.
- Add batch evaluation over bandwidth/latency sweeps.
- Consider more advanced schedulers for larger DAGs.
- Improve timeline semantics with explicit queue-wait and data-ready intervals.
- Explore network reordering policies beyond deterministic topological-source FIFO.

Questions?